DocQA 答辩讲稿

> 配套 Deck：`slides/DocQA-项目汇报.pdf`（10 页）· 建议总时长 8 – 10 分钟 + 3 分钟 Q&A
>
> 讲稿按「一页一屏、一句一个落点」组织；方括号内为动作提示。

第 1 页 · 封面与摘要（约 60 秒）

【停顿 1 秒，让评委看标题】

各位评委好，我是来自转发开发部的井永乐，作品是《基于 AI Agent 的个人本地多模态文档智能问答系统》。

先讲一个大家都遇到过的场景：一份技术文档，你明明记得里面有一张三层架构图，记得某个叫 `FE_HEART_Start` 的参数，但就是想不起文件名，也搜不到——因为搜索引擎只搜标题，不搜正文，更不搜图片。

我的创意就是把「找文件」变成「问内容」：用 **doc2md** 把多模态文档解析成结构化 Markdown，用 **DocBase** 建三路混合检索的知识库，再用 **AGENTS.md** 教会 CodeAgent「先查资料、再回答、必须溯源」。

一句话：**doc2md 让图变成文，DocBase 让文变成证据，AGENTS.md 让 Agent 学会查资料。**

第 2 页 · 系统总体架构（约 75 秒）

【手指三层】

系统是三层解耦：

第一层，文档解析层 doc2md。 输入 PDF、Word、PPT，做格式转换、OCR 提图、VL 多模态解析，最后生成结构化 Markdown；每张图都带「文档名 + 页码 + 图题」的溯源信息，并保留回原文档的链接。

第二层，知识库层 DocBase。 对 Markdown 做语义切片、向量化，建三路索引：稠密向量、BM25、BGE-M3 稀疏；查询时三路召回，RRF 融合，Reranker 精排，输出带溯源元数据的 TOP-K。

第三层，Agent 消费层。 `AGENTS.md` 是知识入口，告诉 Agent 库在哪里、什么时候查、怎么引用；Agent 做意图理解、调 DocBase、用 GLM-5.1 综合推理，最后输出结构化答案和溯源。

三层各司其职，接口清晰：**解析入库 → 混合检索 → Agent 问答。**

第 3 页 · 逻辑视图（约 90 秒）

【按箭头顺序，不要逐字念】

这一页讲「一次问答在系统里怎么流动」。

用户从 CLI 或 Web 进来，Agent 先做意图分析：是命令、流程、版本还是排障。然后调 DocBase 做三路并行召回——注意，三路是互补的：纯向量对 `SD5993` 这种精确型号容易模糊命中，纯关键词又理解不了「分机转发是怎么走的」，所以三路都要。

召回后 RRF 融合，Reranker 精排，取 TOP-K；Agent 做一次「充足性自评」，不够就自动发起第二轮检索，最多两轮。证据够了，GLM-5.1 做跨文档综合，流程类问题重绘 Mermaid，最后输出「结论 + 依据 + 溯源」。

右下角是模型服务，全部由装备部提供：VL 用 Qwen3.6-27B-VL，Embedding 用 Qwen3-Embedding-8B，Reranker 用 Qwen3-Reranker-8B，Chat 用 GLM-5.1。端点可配置、可替换、可 Mock，**数据不出本机**。

第 4 页 · 开发视图（约 75 秒）

【指目录树，再指依赖箭头】

这一页回答「工程上怎么保证可维护」。

左边是目录：五个变更点对应五个目录——doc2md、docbase、agent、service、web。每个目录有自己的 `models.py`，跨层只通过契约通信，字段只增不改。

中间是分层依赖，自上而下单向：入口层 → Agent → 检索 → 解析 → 模型端点。**禁止反向依赖**，所以换 Agent、换模型，不改约定与契约。

右边是核心接口和 `AGENTS.md` 的四类约定：存在性声明、查询策略、溯源强制、产出形式。

可测性也是设计出来的：DocBase 内置 test 模式，输出 recall@k、hit_rate、MRR；四类模型统一注入，单测用 Mock；溯源链路端到端断言。当前 19 项测试全部通过。

第 5 页 · 实施路径（约 70 秒）

【五步手势】

实施是五步渐进：

1 – 3 步是核心闭环，已经完成 Demo：doc2md、DocBase、AGENTS.md。

4 – 5 步是可扩展：后台 Service 做定时/增量更新，Web 做 B/S 问答。

这里想强调一个方法：**全部通过 CodeAgent Vibe Coding 开发，AI 写代码，人做决策。** 人负责问题定义、架构取舍、契约评审和验收；Agent 负责脚手架、解析器、索引器、前端、测试和文档初稿。这也是为什么我一个人能在短周期内把五步都跑通。

1 – 3 步构建核心闭环，4 – 5 步面向团队与领域规模演进。

第 6 页 · doc2md（约 75 秒）

【指流水线，再指「30% – 50%」】

doc2md 解决的是「图片里的知识不可检索」。

流水线六步：多格式输入 → 格式统一 → 图片提取 → VL 释义 → 描述写入 Markdown → 结构化输出。

关键设计有两个：

第一，**溯源命名**：`文档名_Page图序_Figure标题`。任意一张图都能逆向定位到原文档、页码和图题。

第二，**图文一体**：VL 释义不单独存，而是写回原图引用处，切片后图文还在同一块，检索上下文完整。

为什么它是基石？技术文档中约 30% – 50% 的关键信息以图片承载：架构图、流程图、时序图、状态机。传统 RAG 只切文本，这些信息全部丢失。doc2md 把「看不懂的图」变成「读得懂的文」，让多模态知识第一次进入可检索、可问答的领域。

第 7 页 · DocBase（约 75 秒）

【指 build，再指 query 三路】

DocBase 解决「检索既要懂术语，又要懂语义」。

build 模式：Markdown 切片 → Embedding 向量化 → 稀疏索引构建 → 向量库落盘，全程进度条；支持多文档、分类、增量更新。

query 模式：三路召回——稠密向量负责语义，BM25 负责型号/命令/接口名，BGE-M3 稀疏负责词面权重；然后 RRF 融合、Reranker 交叉编码精排、TOP-K 输出。

为什么必须三路？纯向量对 `SD5993`、`FE_HEART_Start` 这类精确术语容易模糊命中；纯关键词又无法理解「分机转发是怎么走的」这类语义问题。三路互补 + RRF 消除量纲差异 + Reranker 精排，才能兼顾语义理解与术语精确。

test 模式用 recall@k、hit_rate、MRR 量化评估，调参不靠感觉。

第 8 页 · AGENTS.md（约 90 秒）

【这页是差异点，放慢】

这一页是本创意最核心的差异点。

RAG 数据库本身不会自动被使用——必须有人「告诉」Agent 它的存在、用法与边界。`AGENTS.md` 就是这座桥梁。

它是什么？CodeAgent 启动时自动加载的项目级指令文件。它声明本仓库有 DocBase 库，约定何时查、怎么查、如何引用，把「人的检索经验」固化成 Agent 可执行的工作流约定。用户不需要学任何命令，像跟同事说话一样提问就行。

右边是 S 形工作流：提问 → 检索 → 综合 → 呈现 → 溯源，全程由约定驱动。Agent 会自主判断「答案够不够好」，不够就发起第二轮检索。

下面四类约定：存在性声明、查询策略、溯源强制、产出形式。特别是「无据不答」——检索不到就明确说不知道，不让模型编。

这就是 DocQA 和传统 RAG 产品的区别：**不是提供一个检索接口，而是让 Agent 养成先查资料的习惯。**

第 9 页 · 演示与交付（约 60 秒，可选）

【如果时间够，现场跑一条命令；否则指截图】

这一页是交付证明：

- `./run.sh demo` 一键跑通 parse → build → ask → eval；
- 5 个示例文档、32 个文本块、3 张图片；
- 8 条测试问题，recall@6、hit_rate、MRR 都是 1.0——**注意，这是演示小样本 + Mock 链路的结果，只证明链路和口径，不代表生产泛化**；
- 19 项测试通过；
- 支持 CLI、Web、watcher 三种入口。

技术上，运行时只依赖 Python 标准库，PDF 可选 pypdf；Web 用标准库 HTTP 服务，默认绑定 127.0.0.1，数据不出本机。

第 10 页 · 结语（约 30 秒）

【收束】

DocQA 不追求做一个更大的模型，而是把已有的模型能力组织成一条**可解释、可核对、可维护**的知识流水线：

- doc2md 让图片进入检索；
- DocBase 让检索兼顾术语与语义；
- AGENTS.md 让 Agent 养成先查资料的习惯；
- 全链路溯源让每一句回答都能回到原文。

让每一份沉睡的本地文档，都能开口说话。谢谢各位评委。

Q&A 预案

Q1：为什么不用现成的 RAG 框架?

A：现成框架解决的是「检索」这一段，但本项目的难点在前面的多模态解析和后面的 Agent 消费约定。doc2md 的图片溯源、AGENTS.md 的四类约定、无据不答的判定，都需要自己控制。底层检索算法（BM25、RRF）并不复杂，自己实现反而更轻、更可控、更容易替换模型端点。

Q2：Mock 模式指标 1.0，可信吗?

A：不可直接外推。这 8 条问题是围绕 5 篇演示文档设计的，规模小、分布窄，指标只证明「链路闭环 + 指标口径正确」。生产验收必须用部门真实文档重建测试集，并接入真实 Embedding/Reranker 后重跑。我在项目书和技术设计里都明确写了这个边界。

Q3：VL 图片释义准确率怎么保证?

A：三层保障：第一，VL 只做描述，不做结论，结论仍由文本证据支撑；第二，释义写回原图引用处，回答时要求与 Page/Fig 原图对照；第三，图片结论必须能回到原图，人工可复核。后续可以加「原图 vs 释义」的抽检机制。

Q4：数据安全怎么保证?

A：默认 Mock 模式不发起任何网络请求；真实模式只调用配置的内部端点；Web 默认绑定 127.0.0.1；文档、索引、图片全部落在本机 `data/`；API Key 只读环境变量，不写日志、不写前端。没有公网依赖。

Q5：为什么用 AGENTS.md，而不是写在 system prompt 里?

A：system prompt 是运行时拼的，跟着代码走；AGENTS.md 是项目级、可版本化、可评审的文件，CodeAgent 启动时自动加载。它把「检索经验」从某个人的脑子里，变成团队可复用、可 diff、可回滚的资产。换 Agent、换模型，约定不动。

Q6：DOCX 页码不精确怎么办?

A：DOCX 本身不存储固定分页，这是格式限制。我们的做法是：`page_mode=estimated` 明示；图片仍然有页内图序和图题；关键文档优先用 PDF/PPT，页码 exact。回答里不会把估算页码伪装成精确页码。

Q7：规模上来以后性能怎么办?

A：当前是 JSON 索引 + 全量余弦，适合个人/团队小规模。扩展点已经留好：Embedding 批量、增量建库、向量库替换（FAISS/Milvus/pgvector），`Retriever` 接口不变；解析和 VL 调用可以并行/队列化。演示规模 32 块检索约 2 ms，瓶颈在真实模型端点，不在检索本身。

Q8：和公司现有知识库/搜索是什么关系?

A：定位是「个人本地文档的最后一公里」，不是替代企业知识库。它解决的是个人电脑上、未入库、多模态、文件名不可靠的那部分资料；团队化之后可以作为一个轻量前端，接部门知识库或工单系统。

Q9：为什么说这是「AI Agent」而不是「RAG 应用」?

A：因为 Agent 不只是被动检索一次。它会做意图判断、query 改写、充足性自评、多轮补充检索、按约定组织产出，并在证据不足时主动拒答。AGENTS.md 是它的工作流约定，DocBase 是它的工具，溯源是它的输出纪律。

Q10：这个项目最大的创新点是什么?

A：如果只选一个，我选 **AGENTS.md 这层「知识消费约定」**。doc2md 和 DocBase 是能力，AGENTS.md 是把能力变成习惯的机制。它让 RAG 从「一个可选接口」变成「Agent 的默认工作方式」，也让检索经验可以像代码一样被评审和迭代。